Agentic Verified Migration Harness

AutoSE Course Project Mid Evaluation Report

AV Aditya: 2024111031 Mohit Sriram: 2024101101 Nandini Chakaravarthy: 2024111034

1. Problem Statement

COBOL still powers a large part of the world's critical systems. Developed in 1959, COBOL continues to be used in the core systems of banks, insurance companies, government organisations, and airlines. A widely cited 2017 analysis estimated that there are around 220 billion lines of COBOL code still in use. Overall, clearly, COBOL is behind approximately 43% of banking systems, 80% of in person card transactions, and 95% of ATM card swipes.

Why does this code need to be migrated?

- **People:** Due to how old and niche the language is, any engineers who have experience working with COBOL are approaching retirement, and relatively few new developers are learning the language.
- **Cost and flexibility:** Mainframe hosting and licensing can be expensive. Older COBOL applications are also difficult to modify, test, and integrate with modern services.
- **Loss of business knowledge:** Important business rules, such as how interest is calculated or how payroll is processed, are often present only in the COBOL source code. In many cases, the documentation is either missing or outdated, and people avoid manual migration out of fear that they might end up losing some of the preserved legacy code.

Why is migrating this code correctly so difficult?

- **COBOL handles arithmetic differently.** It uses fixed-point decimal numbers defined through a PICTURE clause. Values that do not fit into the specified field can be silently truncated. ROUNDED follows specific rules, unsigned fields can drop negative signs, and an ON SIZE ERROR condition leaves the target field unchanged. A Java translation that uses double or misses one of these rules can produce incorrect financial calculations, sometimes with a difference of just a cent.

- **Memory layout matters:** REDEFINES allows multiple fields to share the same memory. Files may store numbers using zoned decimal, packed decimal or binary formats. In some cases, output must match byte for byte.
- **Control flow can be difficult to follow.** COBOL programs often use GO TO, PERFORM THRU, and paragraphs that continue into the next paragraph through fall-through behaviour.
- **Different COBOL dialects behave differently.** IBM Enterprise COBOL and GnuCOBOL can produce different results in certain edge cases.

There are other similar issues as well that come with trying to move from COBOL to another language. We are therefore given two ways of dealing with this problem:

1. **Traditional rule-based translators** often generate Java code that is technically correct but difficult to read and maintain. This kind of output is sometimes referred to as *"JOBOL"* because it retains much of COBOL's original structure.
2. **LLMs can generate more readable code**, but there is no guarantee that the translation is correct. They can introduce subtle semantic errors that are difficult to spot because the generated code often looks perfectly reasonable.

Our Approach

We are building an **agentic, verification-first migration harness**. An LLM agent is responsible for generating the Java code, while a **deterministic verifier** checks whether the generated code behaves like the original COBOL program. The verifier also sends **feedback** to the agent whenever it finds an issue.

- **Source language:** COBOL, with GnuCOBOL 3.2 serving as the reference semantics.
- **Target language:** Java for now. Python and Go are planned for later. The ultimate goal would be to create a COBOL to X harness.
- **Programs in scope:** Batch programs that perform computations on working storage, read from and write to the terminal (ACCEPT/DISPLAY), and work with sequential files.
- **Out of scope:** Embedded SQL, CICS, indexed or relative files, SORT/MERGE, and IBM-specific behaviour.

2. Understanding Migrations

Industry Approaches

- **Rule-based refactoring tools (traditional methods):** Tools such as AWS Mainframe Modernization with Blu Age use predefined rules to translate or refactor legacy code. Their output is deterministic, but the resulting code can be difficult to maintain.
- **LLM-assisted migration tools:** IBM WatsonX Code Assistant is one example. It supports COBOL-to-Java migration.

These examples show that LLMs can significantly speed up code migration. However, they still rely heavily on automated checks, such as builds and tests, along with human review to catch errors.

- **Parallel or dual runs:** Another common approach is to run the old and new systems side by side and compare their outputs before completely switching over. This provides an additional layer of safety and has been adapted in our approach as well (in Tier 1, explained ahead).

Research Papers

- Roziere et al., 2020, "Unsupervised Translation of Programming Languages" (TransCoder): Explores neural approaches to translating code between programming languages and evaluates the generated translations using unit tests.
- Pan et al., ICSE 2024, "Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code": Studies the kinds of bugs introduced by LLMs during code translation and shows that these errors occur frequently.
- COBOLEval (2024): A benchmark designed to evaluate LLMs on COBOL-related tasks.

Known Failures and Bottlenecks in Human Driven Migrations

- **TSB Bank, UK (2018):** A migration to a new core banking platform caused approximately 1.9 million customers to lose access to their accounts. Regulators later fined the bank around £48.65 million.
- **Queensland Health payroll, Australia (2010):** The replacement of a legacy payroll system resulted in serious problems, including large overpayments and underpayments. The total cost was reported to be over AUD 1 billion.

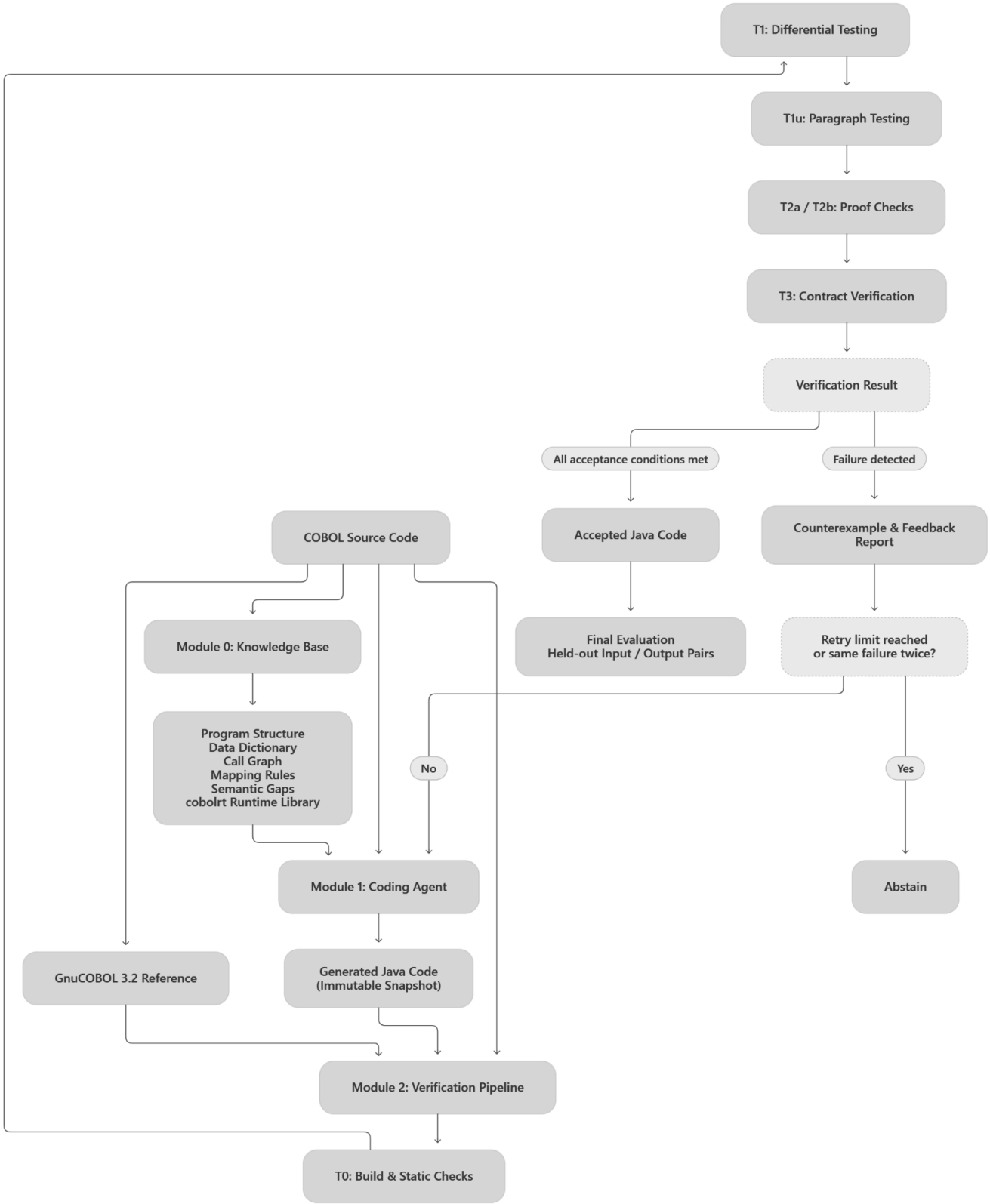
Some of the common problems in such migrations include:

- Loss of important business knowledge that was embedded in the original code.
- Manual testing that covers only common or expected inputs.
- No reliable way to determine whether the generated output is actually correct.
- Switching to the new system all at once, without sufficient gradual validation.
- Small differences in numerical calculations or output formatting that are discovered only after deployment.

Takeaway for our design: The biggest challenge in migration is not necessarily writing the new code. It is making sure that the new code actually behaves like the original. This is the main reason our project has a thorough and formally verified approach.

3. Workflow

1. **Module 0, Knowledge Base:** Designed to parse the COBOL once before any translation and build a read-only knowledge base: paragraph units and what they read and write, field types derived from PIC clauses, the PERFORM call graph, COBOL→Java mapping rules, known semantic gaps, and a shared decimal runtime library. Note: Module 0 is currently a stub, with parsing performed in Module 1.
2. **Module 1, Coding Agent:** An LLM agent that migrates paragraph by paragraph, starting with leaves of the call graph. It uses the knowledge base for context, writes Java, and submits immutable snapshots to the verifier.
3. **Module 2, The Verification Pipeline:** Takes the original COBOL program and the generated Java program, checks whether they behave the same, and reports the level of verification achieved by each unit.
4. **Feedback loop (Module 2 → Module 1):** Whenever the verifier detects a difference, it sends a short, machine readable report that the LLM can use to fix the issue. The report includes:
    - The exact input that caused the failure.
    - The output produced by COBOL and the output produced by Java.
    - The likely reason for the difference (deterministically, no AI yet)
    - The relevant COBOL lines.
5. **The agent uses this feedback** to fix the generated code, after which the verifier runs the checks again. This process continues until the code is accepted, the same failure occurs twice, or the retry limit is reached. If the retry limit is reached, the unit is marked as "abstain" rather than being incorrectly accepted.
6. **Evaluation:** The final code is tested using a separate set of input/output pairs that are never shown to either the agent or the verifier during development.


T1: Differential Testing
T1u: Paragraph Testing
T2a / T2b: Proof Checks
T3: Contract Verification
Verification Result
All acceptance conditions met
Failure detected
Accepted Java Code
Counterexample & Feedback Report
COBOL Source Code
Final Evaluation
Held-out Input / Output Pairs
Retry limit reached or same failure twice?
Module 0: Knowledge Base
Program Structure
Data Dictionary
Call Graph
Mapping Rules
Semantic Gaps
cobolrt Runtime Library
No
Yes
Module 1: Coding Agent
Abstain
GnuCOBOL 3.2 Reference
Generated Java Code (Immutable Snapshot)
Module 2: Verification Pipeline
T0: Build & Static Checks

4. Module 0: Knowledge Base

Role: Module 0 answers one question: "What is true about the COBOL program, the Java target, and the semantic gap between them?" It never writes translations and never judges correctness. Modules 1 and 2 only read from it.

Current state: a working stub of the agreed contract: So that Modules 1 and 2 could be built and integrated without waiting, Module 0 currently ships as a small stub (integration/phase0_stub.py) that implements the agreed query interface over hand-written files in the shared workspace (kb/). It serves three queries:

- toolchain: the pinned compiler and tool versions;
- rules_for: COBOL→Java mapping rules for given constructs, e.g. R-EVALUATE;
- gaps_for: known semantic gaps between COBOL and Java, e.g. G-TRUNCATE for silent high-order truncation, G-OVERFLOW, G-ROUNDING, G-SIGN.

The workspace's kb/manifest.json carries a kb_version.

What already works across modules:

- Every submission from Module 1 records the kb_version it was built on, and Module 2 rejects a submission whose version doesn't match the current knowledge base.
- When the verifier finds a mismatch, its likely cause (e.g. SIGN_LOST) is mapped to the matching gap id (e.g. G-SIGN) and returned as kb_refs. The coding agent's feedback and its pre-translation hazard checklist therefore point at the same rule.
- The shared runtime library cobolrt is owned and validated by Module 2. The knowledge base's copy (kb/target/runtime) must be byte-identical, which Module 2 checks on every submission.

5. Module 1: The Coding Agent

This is the part of the harness that writes Java: it reads a COBOL codebase, decides what to migrate and in what order, and hands code to an independent checker that alone decides whether it is correct.

Three main jobs:

- **Understand.** Work out the structure of a repo it cannot fit in a prompt. Programs, paragraphs, data layouts, who calls whom, who shares state.
- **Decide.** Choose what is worth migrating now, what must wait, and what is out of scope (CICS, SQL, GO TO).
- **Write and repair.** Produce the target code, submit it, and fix it from the verifier's counterexamples.

Why don't standard designs perform well?

Plain ReAct: read, grep, write, build, repeat. ReAct interleaves reasoning with tool calls. With a minimal toolset and an append only context, two major failures occur:

1. Context is filled with raw reads. Every file read adds thousands of tokens. Models use the middle of a long prompt poorly, and a small model with a tokens-per-minute cap hits the wall first. We measured one 20-turn exploration at 68k characters of history.
2. Exploration is blind. The model does not know the architecture, so it greps for guesses. COBOL defeats grep: `COPY` may pull fields in from other files, `PERFORM` chains and fall-through hide control flow, and CICS programs call each other through a variable.

General coding harnesses (Codex-style)

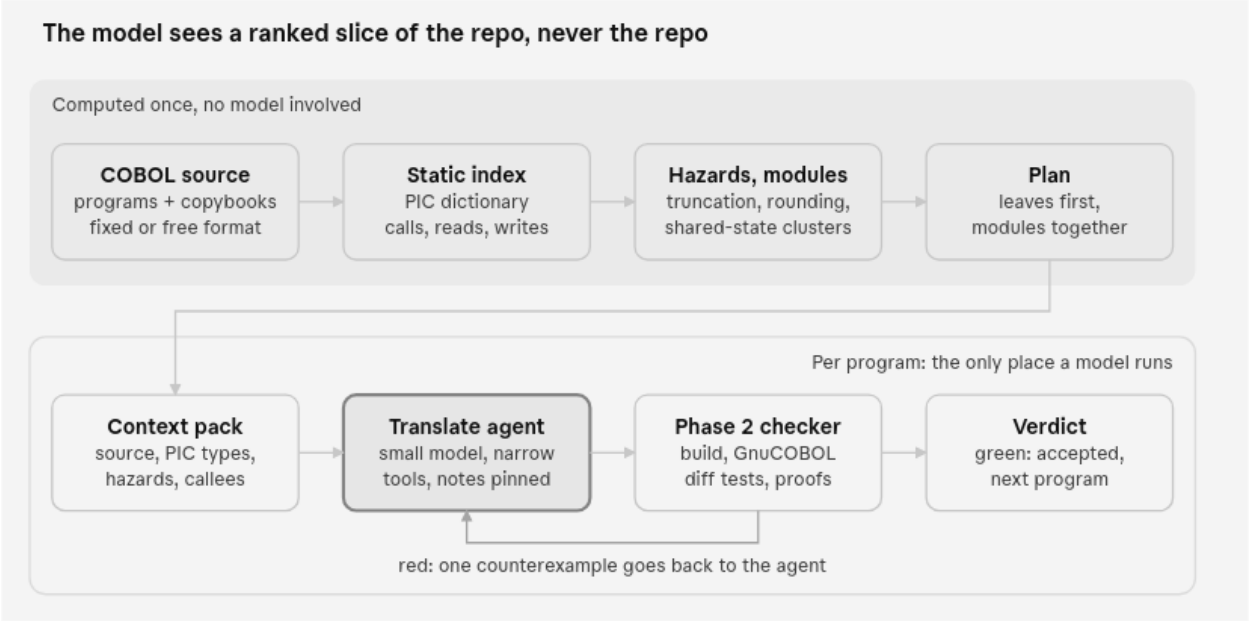
These are built for generic coding tasks: a single repo, its own test suite, one session, a developer reviewing the diff. COBOL migration differs on every axis. Five mismatches:

- **No formal evaluation layer and rich feedback.** Evaluation is usually done through repos own tests. COBOL migration needs both a compiler for syntactic correctness and differential execution for behavioral equivalence. This improves the feedback for the agent helping it with clause-level signals and structured diagnostics. (Efficient fixes ie lowers the needed attempts for fixing)
- **No task decomposition or dependency ordering.** Generic harnesses treat the repo as one job. A COBOL migration is a dependency graph: copybooks before the programs that `COPY` them, called paragraphs before callers, CICS chains in call order etc. Ordering helps plan the sequence of migration and helps delegate independent tasks to concurrent sub agents.
- **No continually updated knowledge base.** Every translation attempt surfaces new facts like a field's true precision, a shared record layout, an implicit control-flow edge. Generic harnesses discard these between sessions. A migration agent needs an append-only knowledge store: discoveries and decisions written after each attempt, read before the next, so the same fact is never re-derived and conflicting decisions do not accumulate.

- **No durability across runs.** A COBOL migration covers thousands of programs and can run for days or weeks. General coding harnesses hold all state in one session context; there is no checkpoints, no job queue, and seamless handling for broken code. A migration agent must be able to stop and restart afresh, knowing exactly which units succeeded, which failed, and which were never attempted.

Approach

We move everything a program can know about the COBOL into a deterministic analyser, and let the model spend its small context on the part only it can do: writing the Java. The model never explores blind; it gets a ranked, budgeted slice of the codebase, a checklist of the traps in it, and exact feedback when it fails.



3.1 A static index replaces blind exploration

A tolerant COBOL analyser (no LLM, 0.8 s for CardDemo) expands `COPY ... REPLACING`, builds a data dictionary from every `PIC` (digits, scale, sign, usage), and per paragraph records reads, writes, `PERFORM`s, `CALL`s, and unsupported constructs — including dynamic targets traced to their `MOVE` or `VALUE`. The agent queries this index through narrow typed tools instead of grepping raw files.

3.2 A hazard scanner turns known traps into a checklist

Before the model writes anything, we scan for the silent differences Pan et al. found and tie each to a knowledge-base gap id with a fix hint:

```
None
[high]   G-TRUNCATE  line 788  MOVE PA-APPROVED-AUTH-CNT
(S9(04)) -> WS-DISPLAY-COUNT (9(03)):
                              high-order digits silently
truncated
         => emulate the PIC width with the runtime decimal
type (truncate, do not round)
[medium] G-OVERFLOW  line 527  ADD into TRAN-CAT-BAL S9(09)V99
without ON SIZE ERROR:
                              overflow is silently truncated
(BigDecimal would keep every digit)
```

On CardDemo: 180 truncation sites, 168 silent-overflow sites, 152 group `MOVE`s, 138 number/text `MOVE`s, 86 `GO TO`s, and 63 sign losses. Each unit gets a risk score; the agent must address every listed hazard before its first submission.

3.3 Modules and a plan, not a pile of paragraphs

Paragraphs sharing a record belong in one migration step. We build a graph (`PERFORM` edges plus shared-write field weights, so global flags don't over-cluster) and run label propagation: CardDemo yields 102 modules and 4 subsystems. A survey agent calls `set_plan` — units leaves-first, modules together, CICS and SQL skipped with a stated reason; a heuristic fallback covers model failure.

3.4 Context packs and context hygiene

Before translating, the agent gets one budgeted **context pack** per program or unit. Each section has its own cap, so the source cannot starve the hazards:

Section	Share of budget	Why it is there
Source	40%	What to translate
Field type table	20%	Exact PIC, digits, scale, sign
Hazards and fixes	22%	The traps, from 3.2
Callees	6%	Already migrated; signatures only
Data producers and consumers	6%	Who feeds this unit and who reads its output (a data-dependence slice, after Weiser's program slicing)
Module mates	6%	Keep naming and types consistent inside a module

Three habits keep history small: tool outputs are clipped at source; old results are elided past a budget while tool-call pairing stays valid (one test: 68k → 15k characters); and a `notes` scratchpad is re-pinned each turn so decisions survive elision.

3.5 The checker closes the loop

Phase 1 submits an immutable snapshot. An adapter runs the formal verifier — build and lint, differential tests against GnuCOBOL, bounded proofs — and converts its output into one short finding per distinct failure. In our test a seeded `PIC S9(2)V9` narrowing produced:

```
None
blocker  behavior_mismatch  VALUE_MISMATCH
stdout:DIVREM.cbl:111:WS-QUOT-R
inputs:   WS-DIVIDEND=12  WS-DIVISOR=1
expected: WS-QUOT-R = +12.0     actual: WS-QUOT-R = +02.0
cobol: DIVREM.cbl:57 DIVIDE WS-DIVIDEND BY WS-DIVISOR
java: Divrem.java:18 (wsQuotR)
kb_refs:  G-OVERFLOW, G-TRUNCATE
```

The same program written correctly came back green in 14 seconds: 7 of 7 paragraphs matched. Because verifier failure causes map to the same gap ids as the hazard scanner, the agent's hints and the checker's evidence point at the same rule.

Small-model robustness is built in: few narrow tools, extra arguments ignored, malformed calls retried, and errors that teach (`"DIVREM is a PROGRAM, not a unit; its units are: ..."`).

4. Future work

The current loop is deliberately one agent working one program at a time. The analysis it already produces points to what comes next.

- **Migrate independent modules in parallel.** The module and subsystem graph shows which parts share no state or calls. Each ready module can go to its own sub-agent with its own context pack and staging directory, scheduled as a dependency graph (a unit becomes ready when its callees are accepted). The checker already verifies per paragraph, so results merge cleanly.
- **A shared migration ledger for long runs.** Replace the free-text notes with a typed, append-only store: the field-to-Java type map, naming conventions, accepted-unit signatures, and approaches that already failed. Sub-agents read it instead of re-deriving it, and a crashed run resumes from the event log, which the agreement already treats as the source of truth.
- **Translation by construction for the solved parts.** The hazards we understand fully (PIC-width truncation, `ROUNDED`, sign handling, `COMP-3`) could be generated from templates over the runtime library, leaving the model only control flow and glue. This removes the largest class of semantic bugs instead of warning about them.
- **Learn from failures.** Cluster counterexamples by signature across runs and store the fixes that worked as retrievable repair recipes. When a failure matches no known gap, emit `kb_gap_reported` so the knowledge-base owner can add a rule for the next run.
- **Route by difficulty.** Use the risk score to send routine paragraphs to a small model and escalate to a larger one after repeated failure on the same signature, keeping most tokens cheap.
- **A real parser and wider scope.** Swap the regex-level analyser for a proper COBOL parser (more exact data flow across `PERFORM THRU` and `REDEFINES`), then add file I/O and CICS as new unit kinds with their own checker support.

- **Measure it.** We have not yet run the ablations that would back these claims: hazard hints on or off, context pack versus raw reads, attempts to green and tokens per unit across model sizes. Those numbers are the next deliverable.

6. Module 2: Verification Pipeline

Given a COBOL program and a Java program that is supposed to do the same thing, the verifier checks as rigorously as the available tools allow whether their behaviour matches. If it finds a difference, it also provides a concrete example showing exactly where the two programs disagree.

What it does NOT do: The verifier does not translate the code (that is Module 1's responsibility), and it does not use an LLM internally. It is completely deterministic, meaning that the same inputs will always produce the same report. We use a tiered system of checks for the same.

Core Principles (Why the Results Can Be Trusted)

- The reference is always real COBOL. Expected outputs are obtained by running GnuCOBOL 3.2, or from our own COBOL model after that model has been checked against GnuCOBOL. We never use any generated Java code or its comments as the source of expected results.
- Every check has an explicit status. Each check returns one of PASS, FAIL, ERROR, TIMEOUT, NOT_APPLICABLE (with a reason), or SKIPPED.
    - A tool crash is never treated as a PASS.
    - Anything that has not been fully implemented is labelled STUB.
- Every failure comes with a reproducible counterexample. This includes the input, the expected output, the actual output, and a one line command that can be used to reproduce the failure.
- Generated code is treated as untrusted. It runs inside a Docker sandbox with no network access and with CPU, memory, and time limits. The verifier also works on a copy of the code.
- Cheaper checks run first. If a simple test finds a bug in one second, there is no reason to spend several more minutes trying to prove that the code is correct.

Tiers of the Verification Pyramid

T0: Build and Code Checks

- **What it catches:** Basic issues that can be identified without even running the program.
- **How it works:** First, the Java code is compiled, with any compilation errors mapped back to the relevant file and line. Static checks are then performed using **tree-sitter.**
    - Checks whether each COBOL field's actual type, calculated from its PICTURE clause, matches the corresponding Java field type.
    - Checks that float or double is not used for decimal fields representing money.
    - Checks that integer types are large enough to hold the required number of digits.
    - Checks that every COBOL paragraph has a corresponding Java implementation (to prevent hallucination, to ensure soundness and completeness)
    - Checks for forbidden APIs, including network access, reflection, process execution, wall-clock time, random numbers, and catch-all exception handling that silently ignores errors.
    - Checks code integrity, including the use of only allowed dependencies and ensuring that the generated code does not access test or evaluation folders.
    - Anti-cheating: Flags constants in the generated code that match earlier counterexamples but do not appear in the original COBOL code. This could indicate that the generated code is being specially adjusted to pass known test cases.

T1: Side-by-Side Testing Against Real COBOL

- **What it catches:** Differences in behaviour between the original COBOL program and the generated Java code, along with concrete examples of where they occur.
- **How it works:**
    - Reads the COBOL data layout and generates inputs that are likely to expose edge cases. These include boundary values (0, ±1, maximum and minimum values, and all 9s), rounding edge cases, values derived from the program's own IF/EVALUATE conditions, empty and over-length text, files containing 0, 1, or multiple records, and seeded random inputs.

    - Runs both programs inside the sandbox and compares stdout, every output file, and the return code byte for byte.
      There is potential here to expand this aspect to coverage-based fuzzing to reduce the number of tests required.
- **When a difference is found:**
    - The raw bytes are converted back into field values
    - Similar failures are grouped together.
    - The counterexample is simplified to the smallest input that still reproduces the failure.
    - The likely cause is identified, for example, "missing truncation" or "rounding missing". These are always reported as likely causes rather than definite explanations.
    - The COBOL statements responsible for writing the incorrect field are identified.
- It also tracks which COBOL paragraphs were exercised during testing, providing paragraph-level coverage information.

T2a: Exhaustive Proof for Small Programs

- **What it solves:** Ordinary testing checks only selected inputs. This tier checks every possible input within a sufficiently small declared input range.
- **How it works:** If a program has fewer than 20,000 possible inputs, the verifier attempts to test every single one. If all inputs produce matching results, this provides a proof of equivalence for the declared input range.

T2b: Mathematical Proof Per Paragraph / JBMC Model Checker (Partial Coverage)

- **What it solves:** Instead of checking only selected test cases, this tier attempts to prove that an individual COBOL paragraph and its corresponding Java method produce the same result for all possible values.
- **How it works:**
    - The COBOL code is parsed into our own intermediate representation, from which we generate a plain Java reference implementation of the paragraph. This is deterministic, done using traditional methods, similar to the “JOBOL” code that is produced.
    - JBMC is then used to check whether the reference implementation and the generated Java implementation leave every field with the same value.
    - Before trusting the reference implementation, it goes through a two step validation process:

        1. Our COBOL model must produce the same results as real GnuCOBOL for every test case.
        2. The generated reference implementation must match our model on random states.
    - Every proof also goes through sanity checks. We verify that the proof is not trivially true and that the checker is able to detect deliberately introduced bugs.
    - Any counterexample found by the checker is run again on the real programs before it is reported back to the agent.

- **Current result:** Complete proofs have been obtained for a few paragraphs, when tested on various inputs.
- **Why this tier cannot currently be used for most paragraphs:**
    - Around half of the paragraphs perform input/output operations.
    - Multiplication or division involving two variables can cause JBMC to run for more than 15 minutes without producing an answer.
    - Java fields of type String or BigDecimal cannot currently be modelled by JBMC.
    - REDEFINES and a few other COBOL constructs are not supported yet.
- When a paragraph cannot be checked using this tier, it is reported as NOT_APPLICABLE along with the exact reason. This is a real limitation that we will try to fix in the next iteration.

T1u: Per-Paragraph Random Testing

- Runs random inputs through individual paragraphs on both the COBOL and Java sides and compares their results.
- This is relatively inexpensive and helps identify which specific paragraph is responsible when a whole program test fails.

T3: Contracts / OpenJML (Partially implemented)

- **What it solves:** Provides another way to establish correctness for some paragraphs that cannot be handled by T2b. For example, certain test cases (listed in the codebase) receives a T3 PASS even though they cannot use T2b because it involves division.
- **How it works:** Contracts are generated directly from the COBOL code for each paragraph. These specify that:
    - Every input and output must remain within its PICTURE range.
    - Only the fields that the paragraph is supposed to modify can change.

    - For paragraphs without loops, the exact expected result must be produced.
- OpenJML then checks whether the generated Java method satisfies these rules.
- **Why this tier cannot currently be used for most paragraphs:** Few paragraphs are eligible and many don't get a contract at all. No contract is generated for paragraphs that do I/O, paragraphs that use GO TO, PERFORM THRU, REDEFINES or COMP-5 etc. Java that stores numbers as BigDecimal or String (OpenJML can't reason about their values). Loops get weaker contracts. There are many similar issues. This is also a real limitation that we will attempt at fixing in the next iteration.

Runtime Library coboIrt

- A Java helper library that handles COBOL arithmetic, text operations, and DISPLAY formatting in the same way as GnuCOBOL.
- It has been tested against GnuCOBOL using 10,000 random cases, with 0 mismatches.
- Using this library in the generated code reduces translation errors and makes more paragraphs eligible for verification.

Reports and Feedback

- The verifier generates report.json (machine readable), summary.md (for human readers), and individual feedback files for each paragraph that can be sent back to the generation agent.
- Every paragraph is assigned a level and a score. The level indicates how far the paragraph has progressed through the verification process.

Level	Reached when
0	**The code builds and passes all static checks**
1	**The side-by-side tests match the COBOL output**
2	**A proof is obtained (T2a or T2b)**
3	**A proof is obtained and the contracts also pass**

A unit is accepted only when all the following conditions are met: it builds successfully and passes the static checks, all tests match the COBOL output with at least 80% paragraph coverage, and there are no failures in T2 or T3.

Future Work under Verification

- Support for more paragraphs in T2b: Add support for the DIVIDE statement and REDEFINES and other similar issues causing syntaxes.
- A formal proof of the runtime library itself
- Automatic loop invariants for T3: Loops currently require hints. A hook is already available for the agent to propose these invariants.
- Forming similar pipelines for other languages, maybe attempt making the verification pipeline itself language agnostic. We will focus on Go and Python for the next iteration.
- Overall the goal would be to adapt to more kinds of paragraphs and making each of the Tiers stronger.

Honest Limitations

- Passing T1 provides strong testing evidence, but it is not a proof that the program is correct for every possible input.
- Our reference is GnuCOBOL, not IBM mainframe COBOL.
- Symbolic proofs through T2b currently cover only a small portion of real world programs. Most of our verification evidence comes from T1, with T2a and T3 providing additional guarantees wherever they apply.

7. Evaluation

Since a huge part of the harness is the verifier, the evaluation metrics for this iteration are mainly about the verification pipeline. We supplied a bunch of input COBOL, Java programme pairs and ran the pipeline on these datasets.

What	Result
False positives on correct translations	**0** across all 14 corpus programs (101 paragraphs)
Test inputs per program (T1)	**248 - 661** (6,387 in total), all byte-identical to GnuCOBOL; 0 excluded

Coverage of the COBOL by those inputs	Paragraph coverage **100% on 13 of 14 programs** (INVENTORY 87.5%, 1 paragraph never reached); statement coverage **86–100%, mean 97%**
T2a exhaustive proofs	**3 of 14 programs** (DIVREM, TAXCALC, TEMPCONV)
T2b complete JBMC proofs	**3 paragraphs** (DIVREM and INVENTORY, SET-RETURN, PAYROLL SPLIT-HOURS)
T3 contract proofs	Pass on **7 of 14 programs**

The planted bugs are COBOL style mistakes deliberately introduced into otherwise correct Java code. These include:

- Removing ROUNDED or using the wrong rounding mode.
- Removing truncation.
- Losing the sign of a number.
- Shifting the decimal point.
- Introducing an off-by-one error in a loop bound.
- Reversing a comparison.
- Padding text on the wrong side.
- Ignoring ON SIZE ERROR.
- Using double instead of a decimal type.
- Swapping two branches.

Basic end to end evaluation of the system has been done using Groq inference provider to check the workings of every module and their integration results.

Rigorous End to end evaluation on a range of tasks of the entire system will be done as a part of our next iteration. As of now, an end-to-end path runs with Module 1, Module 2 and a stub of Module 0 (Java); we tested it on a few simple cases.

8. GitHub

https://github.com/moonIsHere11/autoSE-project-